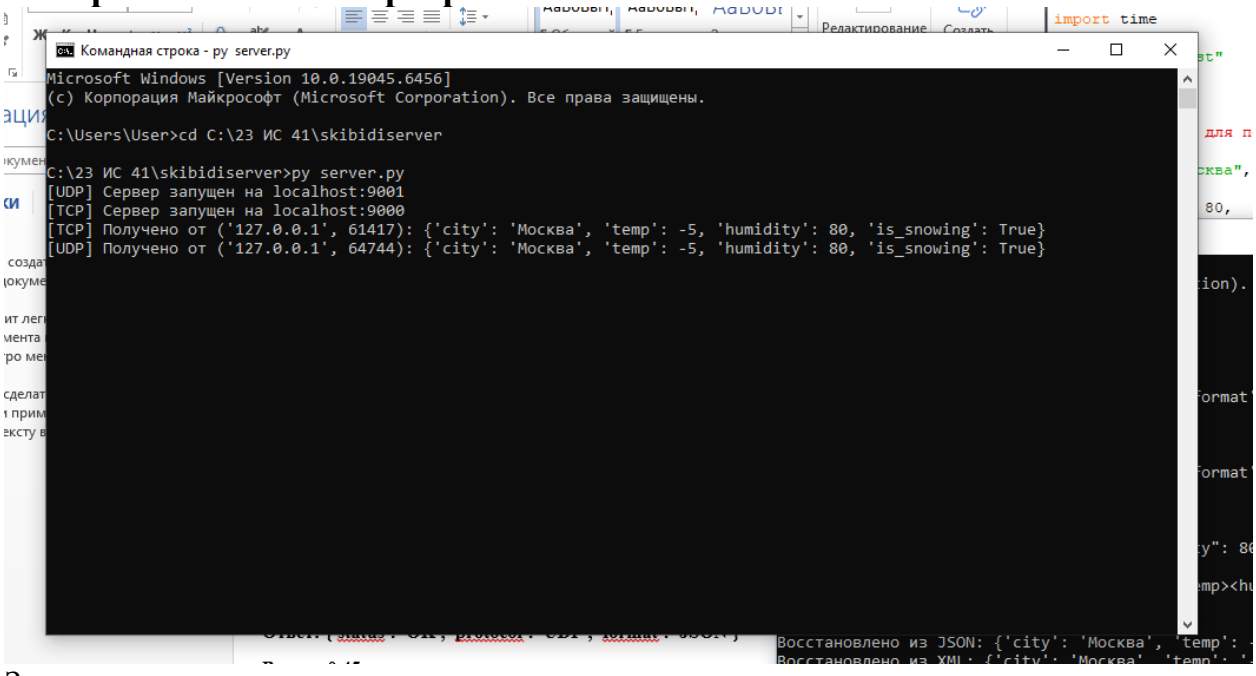
Практическая работа №2

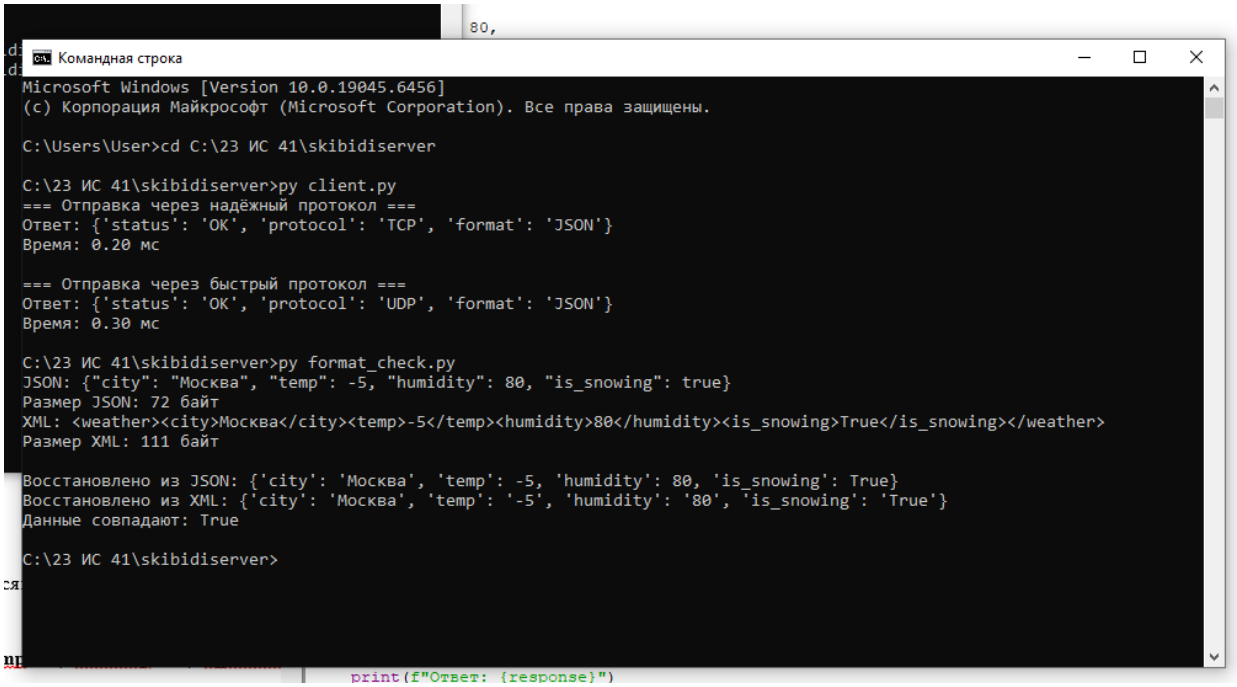
1. Ответы на вопросы

1. Ответ 1. В функции send_reliable создаётся сокет с типом SOCK_STREAM и вызывается connect(). Это TCP, потому что SOCK_STREAM — потоковый сокет, а connect() устанавливает соединение. TCP гарантирует доставку и порядок данных.
2. Ответ 2. В функции send_fast создаётся сокет с типом SOCK_DGRAM и вызывается sendto(). Это UDP, потому что SOCK_DGRAM — датаграммный сокет, соединение не устанавливается. UDP быстрее, но без гарантии доставки.
3. Ответ 3. Данные передаются в формате JSON. В коде это подтверждает строка: client.send(json.dumps(message, ensure_ascii=False).encode("utf-8")). На сервере эти данные разбираются через json.loads(data.decode("utf-8")).
4. Ответ 4. Если отправить строку "hello" (без JSON-кавычек), сервер попробует выполнить json.loads("hello"). Возникнет json.JSONDecodeError, сервер выведет сообщение об ошибке и отправит ответ {"status": "ERROR", "reason": "invalid JSON"}. Для TCP после этого соединение закроется.

2. Скриншоты окна сервера и клиента



Запустив server.py мы видим:
[TCP] Сервер запущен на localhost:9001
[UDP] Сервер запущен на localhost:9000
Значит два сервера запущены на разных портах 9001 и 9000 соответственно



Описание. client.py при запуске отправляет данные о погоде на сервер с начало через надежный протокол(TCP) потом через быстрый протокол(UDP)

3. Тестирование

1. Случай 1. Сервер выключен, клиент TCP:
   При выключенном сервере на строке client.connect((HOST, PORT_R)) возникла ошибка ConnectionRefusedError. Это происходит потому, что TCP сначала требует установить соединение, а сервер не слушает порт.
2. Случай 2. Сервер выключен, клиент UDP:
   UDP-клиент отправляет датаграмму через sendto() и ждёт ответ примерно 2 секунды, потому что client.settimeout(2.0). После таймаута вернётся ответ {'status': 'TIMEOUT'}. UDP не проверяет наличие сервера, поэтому клиент просто ждёт ответ.
3. Случай 3. Клиент отправляет не JSON:
   В исходном client.py если заменить строку отправки на client.send(b"hello"), то сервер попытается разобрать hello как JSON, получит JSONDecodeError, выведет в консоль сообщение об ошибке и отправит клиенту {"status": "ERROR", "reason": "invalid JSON"}.
4. Случай 4. Сравнение форматов:
   JSON компактнее XML: 72 байта против 111 байт для объекта weather. JSON сохраняет типы данных (число, логическое значение), а XML после восстановления даёт строки. Поэтому JSON удобнее для простых данных.

4. Вывод

Для этого приложения я бы выбрал TCP + JSON. TCP нужен для надёжной доставки запроса и получения ответа. JSON компактнее XML. Так как UDP как показано на скриншоте проигрывает даже в скорости TCP выбирать UDP абсолютно не имеет смысла для этого приложения. JSON в свою очередь компактнее XML и поэтому я бы выбрал его для этого приложения.